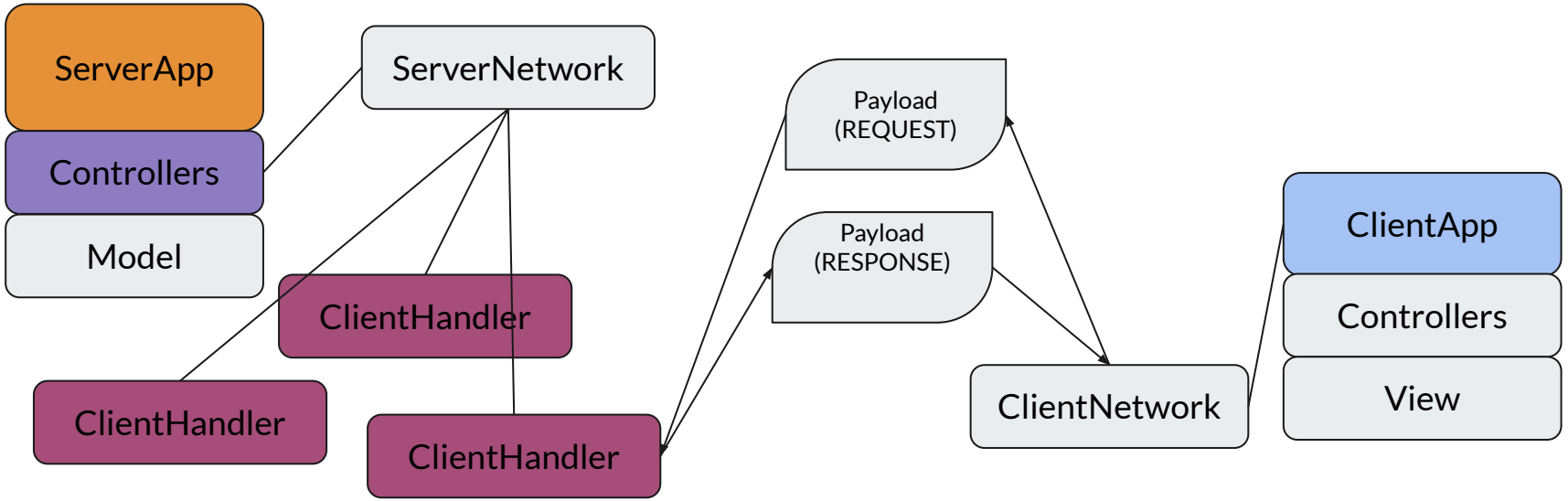
Risiko!

Progetto PAJC 2026

MVC (server & client)



Server (network)

```java
public class ServerNetwork {

    private AtomicLong NEXT_CLIENT_ID = new AtomicLong(1);

    private ServerLobbyController serverLobbyController;
    private ServerGameController serverGameController;
    private BotBrain botBrain;

    private final ExecutorService clientHandlers = Executors.newCachedThreadPool();

    private final Map<Long, ServerClientHandler> connectedClients = new ConcurrentHashMap<>();

    private boolean shuttingDown = false;

    Logger logger = new Logger("SN");

    public ServerNetwork(ServerLobbyController serverLobbyController,
                         ServerGameController serverGameController) {
        this.serverLobbyController = serverLobbyController;
        this.serverGameController = serverGameController;
        logger.log("initiated");
    }
```

Server (network)

```java
/**
 * accept socket connections from client
 * @param serverSocket
 * @throws IOException
 */
public void acceptClient(ServerSocket serverSocket) throws IOException {
    logger.log("waiting for clients");
    while (true) {
        Socket clientSocket = serverSocket.accept();
        clientSocket.setKeepAlive(true);
        logger.log("client socket connected | " + clientSocket.getInetAddress());
        clientHandlers.submit(new ServerClientHandler(clientSocket, this));
    }
}
```

Server (network)

```java
/**
 * receive payload and manage the payload type
 * . LOBBY_REQUEST
 * . GAME_REQUEST
 * @param payload
 * @param sender
 */
public void receive(Payload payload, ServerClientHandler sender) {
    logger.log("payload received → " + payload);
    switch(payload.getType()) {
    case Payloads.LOBBY_REQUEST → receiveRequest(payload, sender, serverLobbyController);
    case Payloads.GAME_REQUEST → receiveRequest(payload, sender, serverGameController);
    case Payloads.SIMPLE_REQUEST → receiveBaseRequest(payload, sender);
    }
}
```

Server (network & controller)

```java
@SuppressWarnings("unchecked")
public <T extends ServerBaseController<T>> void receiveRequest(Payload payload,
            ServerClientHandler sender,ServerBaseController<T> controller) {
    logger.log("request directed");
    try {
    Request<T> request = (Request<T>) payload.getData();
    // pass a lambda to the controller instead of only the request
    // this will work as follow
    // * serverLobbyController will run handle() on the lambda function
    // * which will run request.handle() running the actual request
    // * then building response payload and sending it to the client through the ServerClientHandler
    controller.submitRequest(ctx -> {
        request.handle(ctx); // handle request inside controller

        Payload response = (ctx.buildResponse((BaseRequest) request)); // let controller build response

        logger.log("response built -> " + response);

        if (response.getData() != null) {
            switch (response.getType()) {
            case Payloads.LOBBY_RESPONSE -> handleLobbyResponse(response, sender);
            case Payloads.GAME_RESPONSE -> handleGameResponse(response, sender);
            default -> logger.log("RESPONSE TYPE IS THIS [NOT MANAGED]: " + response.getType());
            }
        }
    });
    } catch (Exception e) {
        e.printStackTrace();
    }

}
```

Server (controller)

```java
public abstract class ServerBaseController<T extends ServerBaseController<T>> implements Runnable {

    protected static final AtomicLong NEXT_ID = new AtomicLong(1);

    protected final BlockingQueue<Request<T>> requests = new LinkedBlockingDeque<>();
    protected boolean requestHandledSuccessfully;

    protected boolean running;


    public abstract void submitRequest(Request<T> request);
    public abstract Payload buildResponse(BaseRequest request);

    @Override
    @SuppressWarnings("unchecked")
    public void run() {
        this.running = true;
        while(this.running) {
            try {
                Request<T> request = requests.poll(500, TimeUnit.MILLISECONDS);
                if (request != null) {
                    request.handle((T) this);
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
```

Server (controller) (lobby)

```java
public class ServerLobbyController extends ServerBaseController<ServerLobbyController> implements Runnable {

    private final ServerLobbyModel model;

    private Lobby cachedLobby;

    private Logger logger = new Logger("SLC");

    // #### CONSTRUCTOR ##################################

    public ServerLobbyController(ServerLobbyModel model) {
        this.model = model;
        logger.log("initiated");
    }

    // #### METHODS #####################################

    public void submitRequest(Request<ServerLobbyController> request) {
        this.cachedLobby = null; // (cachedLobby =/= null) is used as success check
        this.requestHandledSuccessfully = false; // check if request has been successfully executed
        this.requests.offer(request);
        logger.log("request submitted");
    }
```

Server (controller) (lobby)

```
public Payload buildResponse(BaseRequest request) {
    request = (LobbyRequest)request;
    RequestInstruction instruction = request.getInstruction();
    // payload type | updated lobby | requestHandledSuccessfully
    logger.log("about to build response");
    boolean handled = this.requestHandledSuccessfully;
    switch(instruction) {
    case LOBBY_ADD_PLAYER -> {return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_REMOVE_PLAYER -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_READY_PLAYER -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_CREATE -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_DISMANTLE -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_UPDATE_PLAYER -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_ADD_SPECTATOR -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_REMOVE_SPECTATOR -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_ADD_BOT -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    case LOBBY_REMOVE_BOT -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    // devs
    case DEV_CHANGE_LOBBY_CODE -> { return new Payload(Payloads.LOBBY_RESPONSE, this.cachedLobby, handled);}
    default -> {
        logger.log("instruction: " + instruction + "; cannot build response");
        return new Payload(Payloads.ERROR, "cannot build response");
        }
    }
}
```

Server (controller) (lobby)

```
/**
 * add player to lobby
 * @param code
 * @param player
 */
public void addPlayer(Lobby lobby, Player player) {
    try {

        if (lobby.isFull() || lobby.isLocked()) {
            cacheLobby(null);
            markSuccess(false); // sets requestHandledSuccessfully = false
            return;
        }

        model.addPlayer(lobby, player);
        cacheLobby(lobby);
        markSuccess();
        logger.log("player: " + player + "; added to lobby: " + lobby);
    } catch (Exception e) {
        markSuccess(false);
    }
}

    // overloading
    public void addPlayer(String code, Player player) {
        Lobby lobby = model.getLobby(code);
        addPlayer(lobby, player);
    }

    // overloading
    public void addPlayer(long id, Player player) {
        Lobby lobby = model.getLobby(id);
        addPlayer(lobby, player);
    }
```

Server (controller) (game)

```java
public class ServerGameController extends ServerBaseController<ServerGameController>{

    private final ServerGameModel gameModel;
    private final ServerLobbyModel lobbyModel;

    private Game cachedGame;

    private Logger logger = new Logger("SGC");

    public ServerGameController(ServerGameModel gameModel, ServerLobbyModel lobbyModel) {
        this.gameModel = gameModel;
        this.lobbyModel = lobbyModel;
        logger.log("initiated");
    }

    @Override
    public void submitRequest(Request<ServerGameController> request) {
        this.requests.offer(request);
        this.requestHandledSuccessfully = false;
        logger.log("request submitted");
    }
```

Server (controller) (game)

```
@Override
public Payload buildResponse(BaseRequest request) {
    request = (GameRequest) request;
    logger.log("about to build response");
    // payload type | updated game | requestHandledSuccessfully
    RequestInstruction instruction = request.getInstruction();
    boolean handled = this.requestHandledSuccessfully;
    switch(instruction) {
    case GAME_CREATE -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_DISMANTLE -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_SNAPSHOT -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_DEPLOY_TROOPS -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully)
    case GAME_START -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_ATTACK -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_MOVE_TROOPS -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_TRADE_CARDS -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_PICK_CARD -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_NEXT_PHASE -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessfully);}
    case GAME_REPLACE_WITH_BOT -> {return new Payload(Payloads.GAME_RESPONSE, this.cachedGame, this.requestHandledSuccessful
    default -> {
        logger.log("instruction: " + instruction + "; cannot build response");
        return new Payload(Payloads.ERROR, "cannot build response");
        }
    }
}
```

Server (controller) (game)

```
public void deployTroops(long id, long playerId, String countryId, int troops) {
    try {
        Game game = gameModel.getGame(id);
        Player player = game.getPlayer(playerId);

        // only during setup phase
        if (game.getCurrentPhase() == TurnPhase.SETUP) {
            // during setup: no turn order, anyone can deploy simultaneously
            if (!game.isCountryOwnedBy(countryId, playerId)) return;
            if (player.getAvailableTroops() < troops) return;
            if (troops < 0) return; // no withdrawing during setup
            game.getWorld().getCountry(countryId).addTroops(troops);
            player.removeTroopsToDeploy(troops);
            // if player has no more troops to place, mark ready
            if (player.getAvailableTroops() == 0) {
                game.markSetupReady(playerId);
            }
            cacheGame(game);
            markSuccess();
            return;
        }


        if (!game.isPlayerTurn(playerId) || !game.isValidPhase(TurnPhase.DEPLOY)) return; // va

        // withdraw lock logic
        if (troops < 0) {
            String lastCountry = game.getDeployedCountry(playerId);
```

Network Model (shared)

```java
// functional interface
public interface Request<T>{
    public void handle(T ctx);
    public String toString();
}
```

```java
public abstract class BaseRequest implements Serializable{

    protected RequestInstruction instruction = null;

    public BaseRequest(RequestInstruction instruction) {
        this.instruction = instruction;
    }

    // make this to make work the request.handle() line in server network receiveRequest()
    public abstract void handle(ServerBaseController controller);

    public RequestInstruction getInstruction() {
        return this.instruction;
    }

}
```

Network Model (shared)

```
public class SimpleRequest extends BaseRequest {

    public SimpleRequest(RequestInstruction instruction) {
        super(instruction);
    }

    @Override
    public void handle(ServerBaseController controller) {

    }

}
```

Network Model (shared)

```java
public class LobbyRequest extends BaseRequest implements Request<ServerLobbyController>, Serializable {

    private static final long serialVersionUID = 1L;

    private long id;
    private String code;
    private Player player;

    // kinda overloading
    public LobbyRequest(RequestInstruction instruction, long id, Player player) {
        super(instruction);
        this.id = id;
        this.player = player;
    }

    // overloading
    public LobbyRequest(RequestInstruction instruction, String code, Player player) {
        super(instruction);
        this.code = code;
        this.player = player;
    }

    // overloading
    public LobbyRequest(RequestInstruction instruction, Player player) {
        super(instruction);
        this.player = player;
    }
```

Network Model (shared)

```java
@Override
public void handle(ServerLobbyController serverLobbyController) {
    switch (this.instruction) {
        case LOBBY_ADD_PLAYER -> {
            if (this.code == null) {
                serverLobbyController.addPlayer(id, player); // this is more for internal management
            } else {
                serverLobbyController.addPlayer(code, player); // this is used by the user
            }
        }
        case LOBBY_REMOVE_PLAYER -> serverLobbyController.removePlayer(id, player);
        case LOBBY_READY_PLAYER -> serverLobbyController.togglePlayerReady(id, player);
        case LOBBY_CREATE -> serverLobbyController.createLobby();
        case LOBBY_DISMANTLE -> serverLobbyController.dismantleLobby(id);
        case LOBBY_UPDATE_PLAYER -> serverLobbyController.updatePlayer(id, player);
        case LOBBY_ADD_SPECTATOR -> {
            if (this.code == null) {
                serverLobbyController.addSpectator(id, player); // this is more for internal management
            } else {
                serverLobbyController.addSpectator(code, player); // this is used by the user
            }
        }
        case LOBBY_REMOVE_SPECTATOR -> serverLobbyController.removeSpectator(id, player);
        case LOBBY_ADD_BOT -> serverLobbyController.addBot(id, player);
        case LOBBY_REMOVE_BOT -> serverLobbyController.removeBot(id);
        //  ################## [DEV] [REMOVE] ########################
        case DEV_CHANGE_LOBBY_CODE -> serverLobbyController.changeLobbyCode(id, player.getName());
        }
    }
```

```java
@Override
public void handle(ServerBaseController controller) {
    handle((ServerLobbyController) controller);
}
```

Network Model (shared)

```
public class GameRequest extends BaseRequest implements Request<ServerGameControl

    private static final long serialVersionUID = 1L;

    private String code;
    private long id;
    private long playerId;
    private Player player;
    private String countryId;
    private String secondCountryId;
    private int troops;

    public GameRequest(RequestInstruction instruction) {
        super(instruction);
    }

    public GameRequest(RequestInstruction instruction, long id) {
        super(instruction);
        this.id = id;
    }
```

Network Model (shared)

```java
@Override
public void handle(ServerBaseController controller) {
    handle((ServerGameController) controller);
} // need to override

@Override
public void handle(ServerGameController ctx) {
    switch (this.instruction) {
    case GAME_CREATE -> ctx.createGame(id);
    case GAME_DISMANTLE -> ctx.dismantleGame(id);
    case GAME_SNAPSHOT -> ctx.getGameSnapshot(id);
    case GAME_DEPLOY_TROOPS -> ctx.deployTroops(id, playerId, countryId, troops);
    case GAME_START -> ctx.startGame(id);
    case GAME_ATTACK -> ctx.attack(id, playerId, countryId, secondCountryId, troops);
    case GAME_MOVE_TROOPS -> ctx.move(id, playerId, countryId, secondCountryId, troops);
    case GAME_TRADE_CARDS -> ctx.tradeCards(id, playerId);
    case GAME_PICK_CARD -> ctx.pickCard(id, playerId);
    case GAME_NEXT_PHASE -> ctx.nextPhase(id);
    case GAME_REPLACE_WITH_BOT -> ctx.replaceWithBot(id, playerId, player);
    }
}
```

Network Model (shared)

```java
public class Payload implements Serializable{

    private static final long serialVersionUID = 1L;

    public final String type;
    public final Object data;
    public final Object more;

    private boolean broadcast = false;
    public boolean isBroadcast() {return this.broadcast;}
    public void setBroadcast(boolean value) {this.broadcast = value;}

    public Payload(String type, Object data) {
        this.type = type;
        this.data = data;
        this.more = null;
    }

    public Payload(String type, Object data, Object more) {
        this.type = type;
        this.data = data;
        this.more = more;
    }
```

Network Model (shared)

```java
public class Payloads {
    public static final String LOBBY_REQUEST = "LOBBY_REQUEST";
    public static final String LOBBY_RESPONSE = "LOBBY_RESPONSE";

    public static final String GAME_REQUEST = "GAME_REQUEST";
    public static final String GAME_RESPONSE = "GAME_RESPONSE";

    public static final String ERROR = "ERROR";

    public static final String CLIENT_CONNECTED = "CLIENT_CONNECTED";
    public static final String SERVER_SHUTDOWN = "SERVER_SHUTDOWN";

    public static final String CLIENT_DISCONNECTED = "CLIENT_DISCONNECTED";

    public static final String SIMPLE_REQUEST = "SIMPLE_REQUEST";
    public static final String SIMPLE_RESPONSE = "SIMPLE_RESPONSE";
}
```

```java
public enum RequestInstruction {

    // LOBBY
    LOBBY_ADD_PLAYER,
    LOBBY_REMOVE_PLAYER,
    LOBBY_READY_PLAYER,
    LOBBY_CREATE,
    LOBBY_DISMANTLE,
    LOBBY_UPDATE_PLAYER,
    LOBBY_GET_PLAYER,
    LOBBY_ADD_SPECTATOR,
    LOBBY_REMOVE_SPECTATOR,
    LOBBY_ADD_BOT,
    LOBBY_REMOVE_BOT,
    LOBBY_GET_FREE_COLORS,

    // GAME
    GAME_CREATE,
    GAME_DISMANTLE,
    GAME_START,
    GAME_SNAPSHOT,
    GAME_DEPLOY_TROOPS,
    GAME_ATTACK,
    GAME_MOVE_TROOPS,
    GAME_TRADE_CARDS,
    GAME_PICK_CARD,
    GAME_NEXT_PHASE,
    GAME_REPLACE_WITH_BOT,

    // dev
```

Client (network)

```java
public class ClientNetwork {

    private final Socket socket;
    private final ObjectInputStream in;
    private final ObjectOutputStream out;
    private CompletableFuture<Payload> pendingResponse;
    private long id;

    private boolean listening;

    Logger logger = new Logger("CN");

    public ClientNetwork(String ip, int port, Player player) throws IOException, ClassNotFoundException {
        this.socket = new Socket(ip, port);
        this.socket.setKeepAlive(true);
        this.out = new ObjectOutputStream(this.socket.getOutputStream());
        this.out.flush();
        this.in = new ObjectInputStream(this.socket.getInputStream());
        onConnection(player);
        logger.log("initiated");
    }

    public void onConnection(Player player) throws ClassNotFoundException, IOException {
        Payload payload = (Payload) this.in.readObject();
        this.id = ((long) payload.getData()); // set player id based on server response
        player.setId(id);
    }
```

Client (network)

```
private final Object sendLock = new Object();

public Payload sendAndWait(Payload payload) throws Exception {
        synchronized (sendLock) {
            pendingResponse = new CompletableFuture<>();
            send(payload);
            logger.log("sent payload -> " + payload);
            return pendingResponse.get();
        }
}

public void completePending(Payload payload) {
    logger.log("completing pending payload -> "+payload);
    if (pendingResponse != null && !pendingResponse.isDone()) {
        pendingResponse.complete(payload);
        logger.log("payload completed");
    }
}
```

Client (network)

```
public void serverListener(Consumer<Payload> onPayload) {
    this.listening = true;
    Thread listener = new Thread(() -> {
        try {
            while (listening) {
                logger.log("listening to server");
                Payload payload = read();
                logger.log("received payload -> " + payload);

            // only complete pending for direct responses, not broadcasts
            if (!payload.isBroadcast()) {
                completePending(payload);
            }

            onPayload.accept(payload);
            logger.log("pending done: " + (pendingResponse != null ? pendingResponse.isDone() : "null"));
        }
    } catch (Exception e) {
        e.printStackTrace();
        if (listening) {
            logger.log("connection to server closed due to ERROR in server listening");
            onPayload.accept(new Payload(Payloads.SERVER_SHUTDOWN, null));
        }
    }
});
listener.setDaemon(true);
listener.start();
```

Client (network)

```java
public void send(Payload payload) throws IOException {
    logger.log("sending payload");
    synchronized (this.out) {
        this.out.reset();

        /*
        ObjectOutputStream keeps an internal cache that maps every object
        it has ever written to a handle (basically an ID). When you write
        the same object again, instead of serializing the full object it
        just writes the handle — a tiny reference saying "you already have
        this one". This is an optimization to avoid infinite loops with circular
        references and to save bandwidth when the same object is sent multiple times.

        this is also present server side
        */

        this.out.writeObject(payload);
        this.out.flush();
    }
}
```

```java
public Payload read() throws ClassNotFoundException, IOException {
    Payload payload = (Payload) this.in.readObject();
    logger.log("reading payload");
    return payload;
}

public void disconnect() {
    try {
        stopServerListening();
        this.socket.close();
        logger.log("socket closed");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Client (controller)

```java
public abstract class ClientBaseController {

    protected final ClientNetwork clientNetwork;

    public ClientBaseController(ClientNetwork clientNetwork) {
        this.clientNetwork = clientNetwork;
    }

    public abstract Payload sendRequestAndWait(RequestInstruction instruction);

    protected boolean isResponseValid(Payload payload) {
        if (payload == null
            || payload.getData() == null) {
            return false;
        }
        return true;
    }

}
```

Client (controller) (lobby)

```java
public class ClientLobbyController extends ClientBaseController{

    private long currentLobbyId;
    private String currentLobbyCode; // [DEPRECATED] → use currentLobbyId instead

    public ClientLobbyController(ClientNetwork clientNetwork) {
        super(clientNetwork);
    }

    @Override
    public Payload sendRequestAndWait(RequestInstruction instruction) {
        try {
            LobbyRequest request = new LobbyRequest(instruction);
            Payload payload = new Payload(Payloads.LOBBY_REQUEST, request);
            return this.clientNetwork.sendAndWait(payload);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }

    // overloading
    public Payload sendRequestAndWait(RequestInstruction instruction, long id, Player player) {
        try {
            LobbyRequest request = new LobbyRequest(instruction, id, player);
            Payload payload = new Payload(Payloads.LOBBY_REQUEST, request);
            return this.clientNetwork.sendAndWait(payload);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
```

```java
/**
 * join lobby request
 * @param code
 * @param player
 * @return
 */
public boolean joinLobby(long id, Player player) {
    Payload response = sendRequestAndWait(RequestInstruction.LOBBY_ADD_PLAYER, id, player);
    if (isResponseValid(response)) {
        setCurrentLobbyId(id);
        return true;
    } return false;
    throw new NullPointerException("impossible to join lobby");
}

// [CRUCIAL] overloading
public boolean joinLobby(String code, Player player) {
    Payload response = sendRequestAndWait(RequestInstruction.LOBBY_ADD_PLAYER, code, player);
    if (isResponseValid(response)) {
        setCurrentLobbyId(((Lobby)response.getData()).getId());
        return true;
    } return false;
    throw new NullPointerException("impossible to join lobby");
}
```

Client (controller) (lobby)

```java
/**
 * leave lobby request
 * @param player
 */
public void leaveLobby(Player player) {
    sendRequestAndWait(RequestInstruction.LOBBY_REMOVE_PLAYER, getCurrentLobbyId(), player);
}
```

```java
/**
 * toggle ready status request
 * @param player
 */
public void toggleReady(Player player) {
    sendRequestAndWait(RequestInstruction.LOBBY_READY_PLAYER, getCurrentLobbyId(), player);
}
```

Client (controller) (game)

```java
public class ClientGameController extends ClientBaseController {

    private Logger logger = new Logger("CGC");

    public ClientGameController(ClientNetwork clientNetwork) {
        super(clientNetwork);
    }

    @Override
    public Payload sendRequestAndWait(RequestInstruction instruction) {
        try {
            GameRequest request = new GameRequest(instruction);
            Payload payload = new Payload(Payloads.GAME_REQUEST, request);
            return this.clientNetwork.sendAndWait(payload);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
```

Client (controller) (game)

```
public Payload sendRequestAndWait(RequestInstruction instruction, long id, long playerId,
                  String sourceId, String targetId, int troops){
    try {
        GameRequest request = new GameRequest(instruction, id, playerId, sourceId, targetId, troops);
        Payload payload = new Payload(Payloads.GAME_REQUEST, request);
        return this.clientNetwork.sendAndWait(payload);
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }
}
```

Client (controller) (game)

```java
public Game attack(long id, long playerId, String sourceId, String targetId, int troops) {
    Payload response = sendRequestAndWait(RequestInstruction.GAME_ATTACK, id, playerId, sourceId, targetId, troops);
    if (isResponseValid(response)) {
        Game game = (Game) response.getData();
        return game;
    } throw new NullPointerException("impossible to attack");
}

public Game move(long id, long playerId, String sourceId, String targetId, int troops) {
    Payload response = sendRequestAndWait(RequestInstruction.GAME_MOVE_TROOPS, id, playerId, sourceId, targetId, troops);
    if (isResponseValid(response)) {
        Game game = (Game) response.getData();
        return game;
    } throw new NullPointerException("impossible to move");
}


public Game nextPhase(long id) {
    Payload response = sendRequestAndWait(RequestInstruction.GAME_NEXT_PHASE, id);
    if (isResponseValid(response)) {
        Game game = (Game) response.getData();
        return game;
    } throw new NullPointerException("impossible to end turn");
}


public Game tradeCards(long id, long playerId) {
    Payload response = sendRequestAndWait(RequestInstruction.GAME_TRADE_CARDS, id, playerId);
    if (isResponseValid(response)) {
        Game game = (Game) response.getData();
        return game;
    } throw new NullPointerException("impossible to trade cards");
}
```

Client (controller) (view)

```java
public class ClientViewController {

    private final MainFrame mainFrame;
    private final ClientNetwork clientNetwork;
    private final ClientLobbyController clientLobbyController;
    private final ClientGameController clientGameController;
    private final Player player;

    private Game gameSnapshot;

    private ExecutorService networkCommunications = Executors.newCachedThreadPool();

    private boolean readySet = false;
    private boolean isStartedHandled = false;
    private boolean areTurnsStarted = false;

    private TurnPhase oldPhase;

    Logger logger = new Logger("CVC");

    public ClientViewController(MainFrame mainFrame, ClientNetwork clientNetwork, ClientLobbyController clientLobbyController, Clie
        this.mainFrame = mainFrame;
        this.clientNetwork = clientNetwork;
        this.clientLobbyController = clientLobbyController;
        this.clientGameController = clientGameController;
        this.player = player;

        wireView();
    }
```

Client (controller) (view)

```
private void wireView() {
    mainFrame.setSnapshotSupplier(() -> this.gameSnapshot);
    mainFrame.setOnStartGame(this::startGame);
    mainFrame.setOnAddBot(this::addBot);
    mainFrame.setOnRemoveBot(this::removeBot);
    mainFrame.setOnLeaveLobby(this::leaveLobby);
    mainFrame.setOnLeaveLobbyAsSpectator(this::leaveLobbyAsSpectator);
    mainFrame.setOnToggleReady(this::toggleReady);
    mainFrame.setOnLeaveGame(this::leaveGame);
    mainFrame.setOnSurrenderGame(this::surrenderGame);
    mainFrame.setOnLeaveGameAsSpectator(this::leaveGameAsSpectator);
    mainFrame.setOnNextPhase(this::nextPhase);
    mainFrame.setOnColorPicked((slot, color) -> {
        player.setColor(color);
        slot.pick(player.getName());
        networkCommunications.execute(() -> clientLobbyController.updatePlayer(player));
    });
    mainFrame.setOnCardsButton(() -> {
        SwingUtilities.invokeLater(() -> {
            mainFrame.getGameView().showCardsDialog(clientNetwork.getId(), this::onTradeCards);
        });
    });

    mainFrame.setOnOptionChanged(() -> {
        mainFrame.getGameView().resetSelection();
    });
//      mainFrame.setOnTradeCards(() -> {
//
//      });
```

Client (controller) (view) - still in wireView()

```
ClientInputController inputController = new ClientInputController(
    mainFrame.getGameView(), clientNetwork.getId(), clientLobbyController, clientGameController
);

inputController.setOnNotEnoughTroops(mainFrame::showNotEnoughTroops);
inputController.setOnNotBordering(mainFrame::showNotBordering);
inputController.setOnCountryNotOwned(mainFrame::showCountryNotOwned);
inputController.setOnCannotAttackInMovePhase(mainFrame::showCannotAttackInMovePhase);
inputController.setOnCannotMoveInAttackPhase(mainFrame::showCannotMoveInAttackPhase);
inputController.setOnAttackResult(mainFrame::showAttackResult);
inputController.setOnFinishedSetupDeploy(mainFrame::showEndSetup);

mainFrame.setOnCountryClicked(inputController::onInput);
```

Client (controller & network) (view)

```
public void handleServerPayload(Payload payload) {
    logger.log("handling server payload");
    switch (payload.getType()) {
    case Payloads.LOBBY_RESPONSE -> handleLobbyResponse(payload);
    case Payloads.GAME_RESPONSE  -> handleGameResponse(payload);
    case Payloads.ERROR          -> mainFrame.showError("something went wrong");
    case Payloads.SERVER_SHUTDOWN -> {
        mainFrame.showMessage("Server has shut down", "Disconnected");
        mainFrame.jumpToStartUpFrame();
    }
    case Payloads.CLIENT_DISCONNECTED -> {
        mainFrame.showMessage("You have been disconnected", "Disconnected");
        mainFrame.jumpToStartUpFrame();
    }
    }
}
```

```
private MainFrame stepIntoMainFrame() {
    MainFrame mainFrame = new MainFrame();
    ClientViewController viewController = new ClientViewController(
        mainFrame, clientNetwork, clientLobbyController, clientGameController, player
    );
    this.clientNetwork.serverListener(viewController::handleServerPayload); // setting up the consumer
    return mainFrame;
}
```

(in StartUpFrame)

Client (controller & network) (view)

```
private void handleLobbyResponse(Payload payload) {
    logger.log("handle lobby response");
    Lobby lobby = (Lobby) payload.getData();
    mainFrame.updateLobby(lobby);
}

private void handleGameResponse(Payload payload) {
    logger.log("handle game response");
    Game game = (Game) payload.getData();

    // if player got eliminated
    if (!game.getJustEliminated().isEmpty()) {
        for (Player eliminated : game.getJustEliminated()) {
            if (eliminated.getId() == player.getId()) {
                    mainFrame.showGotEliminatedDialog();
                    mainFrame.setSpectatorMode(true);
            } else {
                    mainFrame.showEliminationDialog(eliminated.getName());
            }
        }
        game.clearJustEliminated();
    }

    if (game.isEnded()) {
        logger.log("game ended");
```

Client (controller & network) (view)

```
private void startGame() {
    networkCommunications.execute(() -> {
        this.gameSnapshot = clientGameController.createGame(clientLobbyController.getCurrentLobbyId(), player);
        this.gameSnapshot = clientGameController.startGame(this.gameSnapshot.getId(), player);
        mainFrame.getGameView().updateGameInfo(clientNetwork.getId());
        mainFrame.getGameView().resetSelection();
    });
}

private void addBot() {
    networkCommunications.execute(() -> {
        clientLobbyController.addBot();
    });
}

private void removeBot() {
    networkCommunications.execute(() -> {
        clientLobbyController.removeBot();
    });
}

private void leaveLobby() {
    networkCommunications.execute(() -> {
        clientLobbyController.leaveLobby(player);
        clientNetwork.disconnect();
        mainFrame.jumpToStartUpFrame();
    });
}
```

Client (controller) (view & input)

```java
public class ClientInputController {

    private final ClientGameController gameController;
    private final ClientLobbyController lobbyController;
    private final GameView gameView;
    private final long playerId;

    private ExecutorService networkCommunications = Executors.newCachedThreadPool();

    private Runnable onNotEnoughTroops;
    private Runnable onNotBordering;
    private Runnable onCountryNotOwned;
    private Runnable onCannotAttackInMovePhase;
    private Runnable onCannotMoveInAttackPhase;
    private Consumer<AttackReport> onAttackResult;
    private Runnable onFinishedSetupDeploy;


    public void setOnNotEnoughTroops(Runnable action) { this.onNotEnoughTroops = action; }
    public void setOnNotBordering(Runnable action) { this.onNotBordering = action; }
    public void setOnCountryNotOwned(Runnable action) { this.onCountryNotOwned = action; }
    public void setOnCannotAttackInMovePhase(Runnable action) { this.onCannotAttackInMovePhase = action; }
    public void setOnCannotMoveInAttackPhase(Runnable action) { this.onCannotMoveInAttackPhase = action; }
    public void setOnAttackResult(Consumer<AttackReport> onAttackResult) { this.onAttackResult = onAttackResult;}
    public void setOnFinishedSetupDeploy(Runnable onFinishedSetupDeploy) { this.onFinishedSetupDeploy = onFinishedSetupDeploy;}
```

Client (controller) (view & input)

```
private void handleInfo(ClientInput input, Game snapshot) {
    gameView.getWorldPanel().infoClick(input.getTargetId());
    updateCountryInfo(input, snapshot);
}

private void handleDeploy(ClientInput input, Game snapshot) {
    if (!input.isMouse()) {return;}
    int toDeploy = switch (input.getButton()) {
    case MouseEvent.BUTTON1 -> +1; // left click add troops
    case MouseEvent.BUTTON3 -> -1; // right click remove troops
    default -> 0; // do nothing
    };

    if (toDeploy == 0) return;

    Country country = snapshot.getWorld().getCountry(input.getTargetId());
    Player player = snapshot.getPlayer(playerId);

    // client-side validation before optimistic update
    if (toDeploy > 0 && player.getAvailableTroops() < 1) return; // no troops to deploy
    if (toDeploy < 0 && country.getTroops() <= 1) return;         // can't withdraw below 1


    executeRequest(() -> {
        Game updated = gameController.deployTroops(snapshot.getId(), playerId, input.getTargetId(), toDeploy);
        if (updated != null) {
            SwingUtilities.invokeLater(() -> {
            gameView.getWorldPanel().deployClick(input.getTargetId());
            gameView.update();
            updateCountryInfo(input, updated);
            });
            if (updated.getCurrentPhase() == TurnPhase.SETUP
                    && updated.getPlayer(playerId).getAvailableTroops() <= 0) {
                onFinishedSetupDeploy.run();
            }
        }

    });
```

Model (shared)

```java
public class Player implements Serializable{

    private static final long serialVersionUID = 1L;

    private long id;
    private PlayerColor color;
    private String name;
    private Deck deck;
    private int availableTroops;

    private boolean bot = false;

    public Player(String name, PlayerColor color) {
        this.name = name;
        this.color = color;
        this.deck = new Deck();
        this.availableTroops = 0;
    }
```

```java
public class Lobby implements Serializable {

    public final int MAX_LOBBY_SIZE = 6;
    public final int MIN_LOBBY_SIZE = 1;

    private static final SecureRandom secRandom = new SecureRandom();
    private static final int LOBBY_CODE_LEN = 6;
    private static final String LOBBY_CODE_CHARSET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890";

    private long id;
    private String code;
    private Slot[] slots;
    private List<Player> spectators;

    // when closed the lobby appear inexistent
    private boolean closed;

    // if locked when joining it goes into spectator mode
    private boolean locked;

    public Lobby(long id, String code) {
        this.id = id;
        this.code = code;
        this.slots = generateSlots();
        this.spectators = new ArrayList<>();
    }
```

Model (shared)

```java
public class Country implements Serializable{

    private final String id;
    private int troops;
    private ArrayList<String> bordersIds;
    private String continentId;
    private PlayerColor owner;

    public Country(String id, ArrayList<String> bordersIds, String continentId) {
        this.id = id;
        this.bordersIds = bordersIds;
        this.continentId = continentId;
    }
```

Model (shared)

```java
public class Continent implements Serializable {

    private final String id;
    private int troopsBonus;

    public Continent(String id, int troopsBonus) {
        this.id = id;
        this.troopsBonus = troopsBonus;
    }
```

Model (shared)

```java
public class World implements Serializable {


    // id | country
    private HashMap<String, Country> countries;
    private HashMap<String, Continent> continents;


    public World() {
        this.countries = new HashMap<>();
        this.continents = new HashMap<>();
    }
```

Model (shared)

```java
public class Game implements Serializable{

    private static boolean TEST_MODE_FOR_DEV = true;

    private static final long serialVersionUID = 1L;

    private static final int COUNTRY_TROOPS_CONSTANT = 3; // → troops = N_countries / this_c
    private static final int BASE_TROOPS_PER_TURN = 3;
    private static final double DYNAMIC_TROOP_DENSITY = 1.1;

    private static final boolean CONTINENT_BONUS = false;


    private long id; // the id is the same as the lobby
    private Lobby lobby;
    private World world;
    private Deck deck;
    private boolean started;
    private boolean ended;
    private boolean turnsStarted;
    private Player winner;
    private boolean conqueredFlag; // if a country has been conquered. Used to pick card

    // setup game mechanics
    private static final Map<Integer, Integer> SETUP_TROOPS = Map.of(
            2, 40,
            3, 35,
            4, 30,
            5, 25,
            6, 20);
    private Set<Long> setupReadyPlayers = new HashSet<>(); // unique items

    // turn mechanics
    private List<Player> turnOrder;
    private int currentPlayerIndex;
    private TurnPhase currentPhase;
```

```java
    // elimination mechanics
    private List<Player> justEliminated = new ArrayList<>();


    // deploy mechanics
    private Map<Long, String> deployedCountry = new HashMap<>();   // playerId → countryId
    private Map<Long, Integer> deployedTroops = new HashMap<>();   // playerId → troops deployed this turn


    public Game(Lobby lobby) {
        this.id = lobby.getId();
        this.lobby = lobby;
        this.world = new World();
        this.deck = new Deck();
        this.ended = false;
        this.started = false;
        this.turnsStarted = false;
        this.conqueredFlag = false;
    }
```

Model (server)

```
public class ServerLobbyModel {

    private ConcurrentHashMap<Long, Lobby> lobbies;

    public ServerLobbyModel() {
        this.lobbies = new ConcurrentHashMap<>();
    }
```

Model (server)

```
public class ServerGameModel {

    // when a Game object is created, inside of it a it cr
    private GameData baseGameData;

    // TODO: this is accessed by multiple threads, it shou
    private ConcurrentHashMap<Long, Game> games;


    public ServerGameModel() {

        try {
            this.baseGameData = new GameData();
        } catch (IOException e) {
            // error along loading gameData.json
            e.printStackTrace();
        }

        this.games = new ConcurrentHashMap<>();
    }
```

Model (server)

```java
public class GameData {


    InputStream countriesStream = ServerGameModel.class.getResourceAsStream("/server/resources/gameData.json");
    InputStream continentsStream = ServerGameModel.class.getResourceAsStream("/server/resources/gameData.json");

    // these are READ ONLY
    private HashMap<String, Country> countries;
    private HashMap<String, Continent> continents;

    public GameData() throws IOException {
        this.countries = GameDataLoader.loadCountryMap(countriesStream);
        this.continents = GameDataLoader.loadContinentsMap(continentsStream);
    }

    // copy constructor
    public GameData(GameData base) {

        // deep copy of country
        this.countries = new HashMap<>();
        base.countries.forEach((id, country) -> {
            this.countries.put(id, country.copy());
        });

        // no need to deep copy as the continents are read only and will not be edited
        this.continents = base.continents;
    }

    public HashMap<String, Country> getCountryMap(){
        return this.countries;
    }

    public HashMap<String, Continent> getContinentsMap(){
        return this.continents;
    }
```

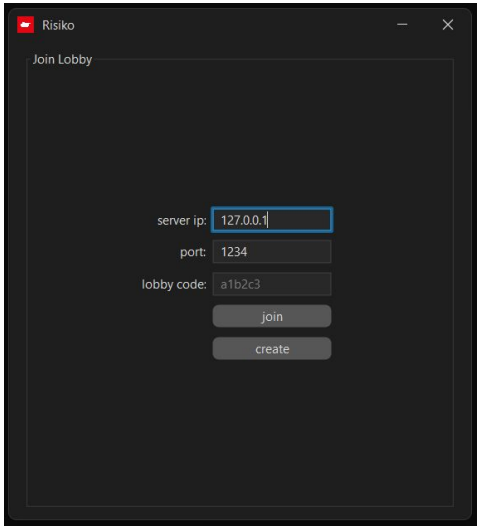
```json
"countries": {
  "afghanistan": {
    "id": "afghanistan",
    "continentId": "central_eurasia",
    "bordersIds": [
      "central_asian_mountains",
      "iran",
      "karakum_desert",
      "pakistan"
    ]
  },
  "alberta": {
    "id": "alberta",
    "continentId": "south_canada",
    "bordersIds": [
      "british_columbia",
      "central_rocky_mountains",
      "north_manitoba",
      "northwest_territories",
      "south_manitoba"
    ]
  },
  "amazonas": {
    "id": "amazonas",
    "continentId": "brazil",
    "bordersIds": [
      "bolivia",
      "colombia",
      "north_brazil",
      "northeast_coast",
      "peru",
      "venezuela",
      "west_central_brazil"
    ]
  },
  "amur": {
    "id": "amur",
```

```json
},
"continents": {
  "alaska": {
    "id": "alaska",
    "bonus": 5
  },
  "australia": {
    "id": "australia",
    "bonus": 5
  },
  "brazil": {
    "id": "brazil",
    "bonus": 5
  },
  "central_africa": {
    "id": "central_africa",
    "bonus": 5
  },
  "central_america": {
    "id": "central_america",
    "bonus": 5
  },
  "central_eurasia": {
    "id": "central_eurasia",
    "bonus": 5
  },
  "east_africa": {
```

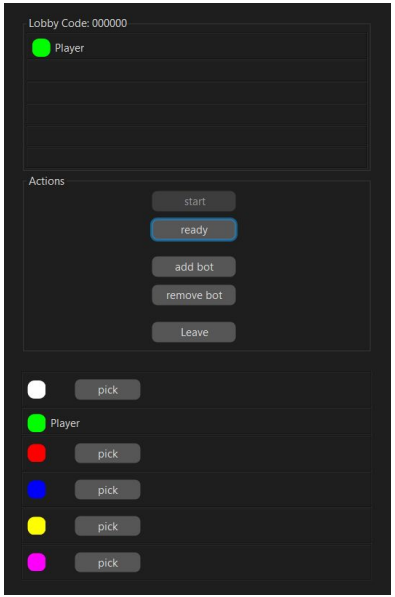
Model (server)

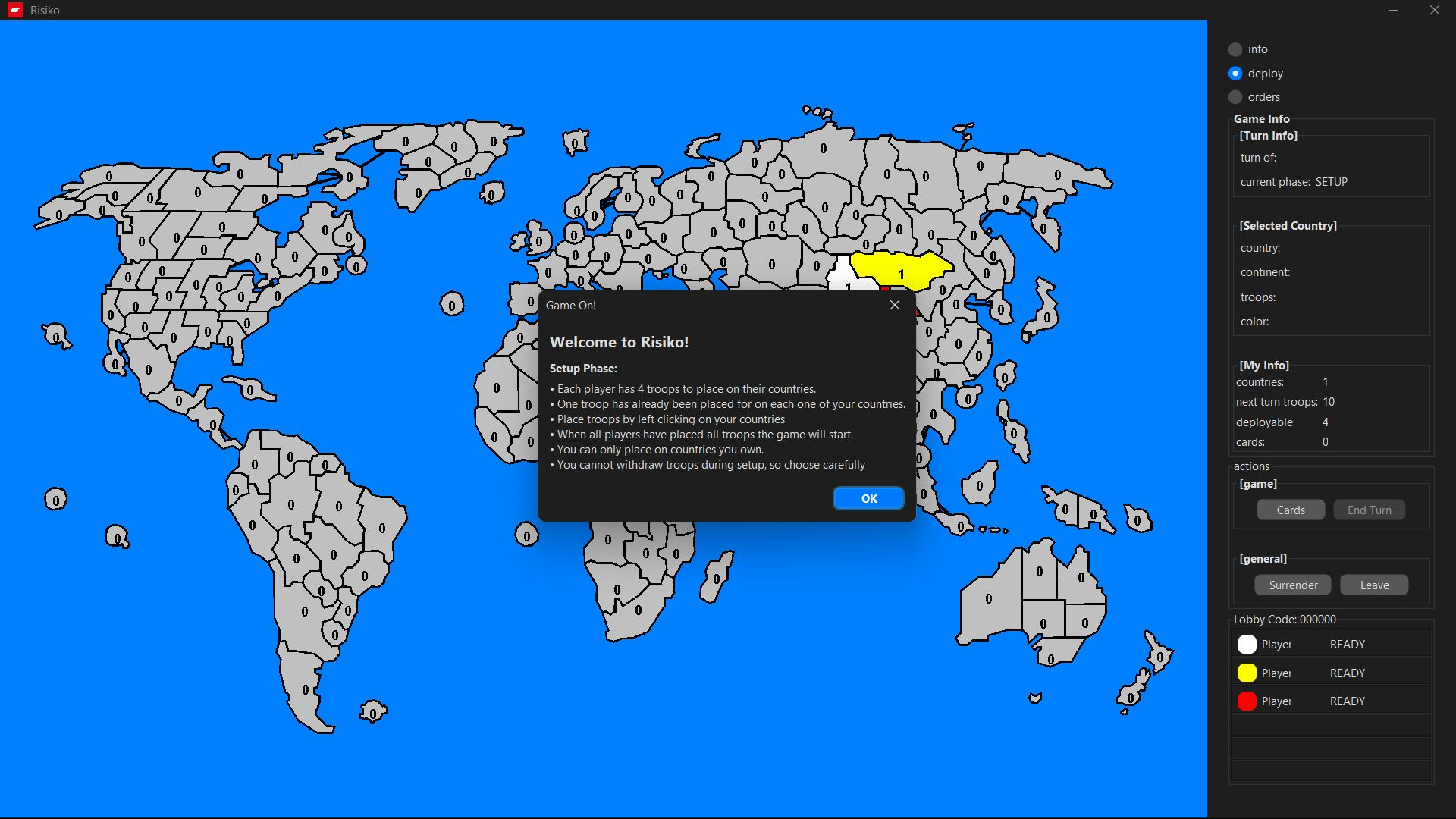
```java
public class BattleProbability {


    public static int[] rawP(int atk, int def) {
        int P = 47;
        int[] losses = new int[] {0,0};
        while (atk>0 && def>0) {
            IntStream.range(0, Math.min(atk, def)).forEach(n -> {
                if (ThreadLocalRandom.current().nextInt(1, 100) <= 47) {
                    losses[1] += 1;
                } else {
                    losses[0] += 1;
                }
            });
        }
        return losses;
    }

    public static int[] bloodDices(int attackers, int defenders) {
        int[] totalLosses = new int[] {0,0};
        int[] losses = dices(attackers, defenders);
        attackers -= losses[0];
        defenders -= losses[1];
        totalLosses[0] += losses[0];
        totalLosses[1] += losses[1];
        while(attackers > 0 && defenders > 0) { // go on until nor the attacker has n
            losses = dices(attackers, defenders);
            attackers -= losses[0];
            defenders -= losses[1];
            totalLosses[0] += losses[0];
            totalLosses[1] += losses[1];
        }
        return totalLosses;

    }
```

View (startUpFrame)

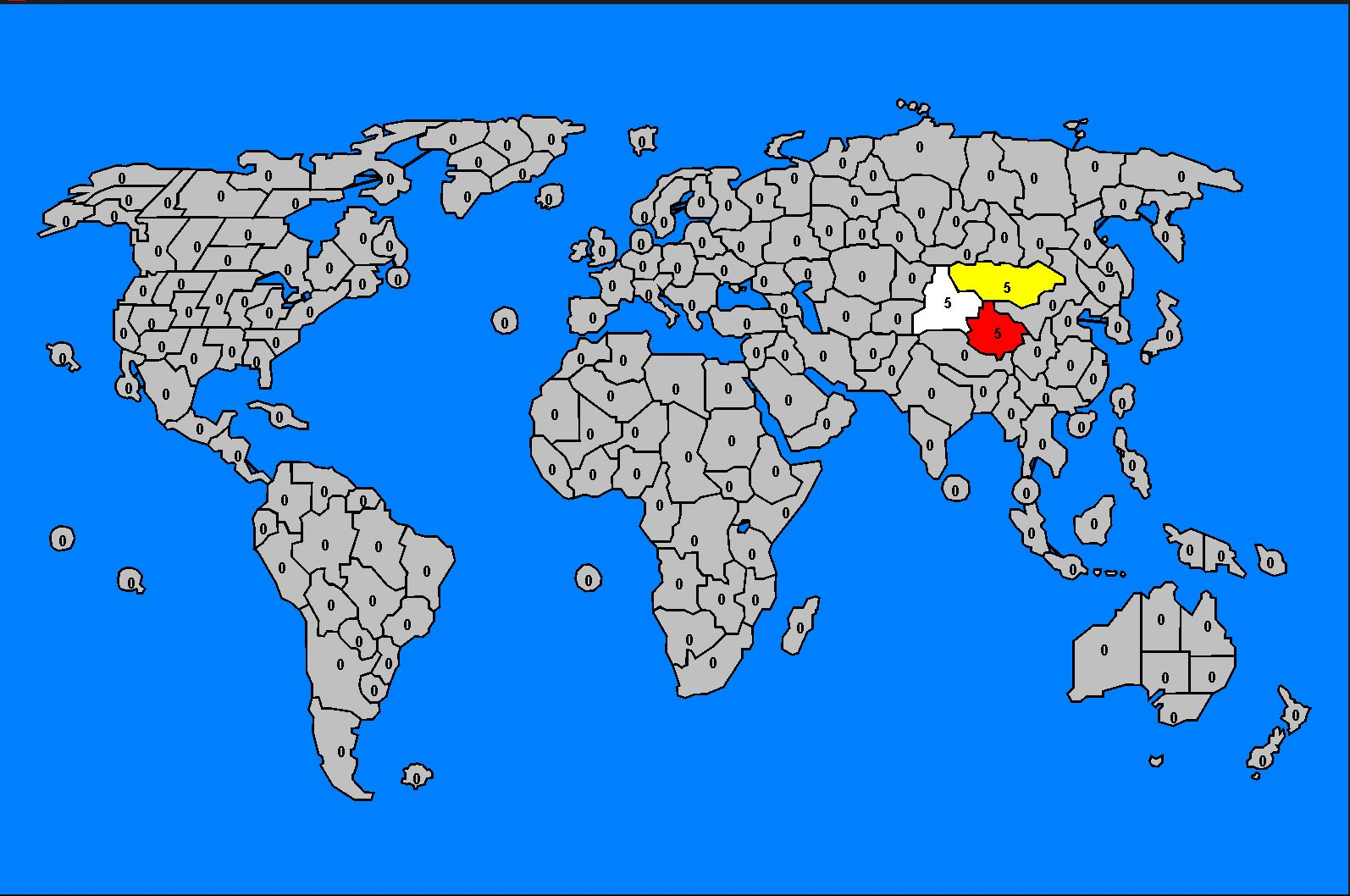


View


Lobby Code: 000000
Player
Actions
start
ready
add bot
remove bot
Leave
pick
Player
pick
pick
pick
pick


Risiko
info
deploy
orders
Game Info
[Turn Info]
turn of:
current phase: SETUP
[Selected Country]
country:
continent:
troops:
color:
[My Info]
countries: 1
next turn troops: 10
deployable: 4
cards: 0
actions
[game]
Cards
End Turn
[general]
Surrender
Leave
Lobby Code: 000000
Player READY
Player READY
Player READY
Game On!
Welcome to Risiko!
Setup Phase:
• Each player has 4 troops to place on their countries.
• One troop has already been placed for on each one of your countries.
• Place troops by left clicking on your countries.
• When all players have placed all troops the game will start.
• You can only place on countries you own.
• You cannot withdraw troops during setup, so choose carefully
OK

Risiko



info
deploy
orders

Game Info

[Turn Info]

turn of: Player

current phase: DEPLOY

[Selected Country]

country:

continent:

troops:

color:

[My Info]

countries: 1

next turn troops: 10

deployable: 0

cards: 0

actions

[game]

Cards | Start Attack

[general]

Surrender | Leave

Lobby Code: 000000

Player	READY
Player	READY
Player	READY

View (Shape & .svg)

```
public class CountryRenderer {

    private static final Color BACKGROUND_COLOR = Color.LIGHT_GRAY;
    private static final Color BORDER_COLOR = Color.BLACK;
    private static final Color HIGHLIGHT_COLOR = new Color(255, 190, 0);
    private static final Color LIGHT_HIGHLIGHT_COLOR = new Color(255, 222, 125);
    private static final Color ARMY_COLOR = Color.BLACK;
    private static final Stroke SELECTED_STROKE = new BasicStroke(3);
    private static final Stroke BASE_STROKE = new BasicStroke(2);

    private static final int TROOPS_CIRCLE_RADIUS = 17; // keep 17
    private static final Font TROOPS_FONT = new Font("Courier", Font.BOLD, TROOPS_CIRCLE_RADIUS - 3);


    private String id;
    private Shape shape;
    private Color fill;
    private Color strike;
    private Stroke stroke;
    private boolean selected;
    private SelectionType selectionType;

    private String name;
```

View (Shape & .svg)

```
public class SvgLoader {


    private static boolean USE_FILL_COLOR = false;
    private static boolean USE_STRIKE_COLOR = true;

    private Document doc;
    private String svgFilePath;

    public SvgLoader(String svgFilePath) {
        this.svgFilePath = svgFilePath;
        try {
            DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
            DocumentBuilder builder = factory.newDocumentBuilder();
            this.doc = builder.parse(new File(this.svgFilePath));
        } catch (Exception e) {
            // error opening/parsing file
            e.printStackTrace();
        }
    }
```

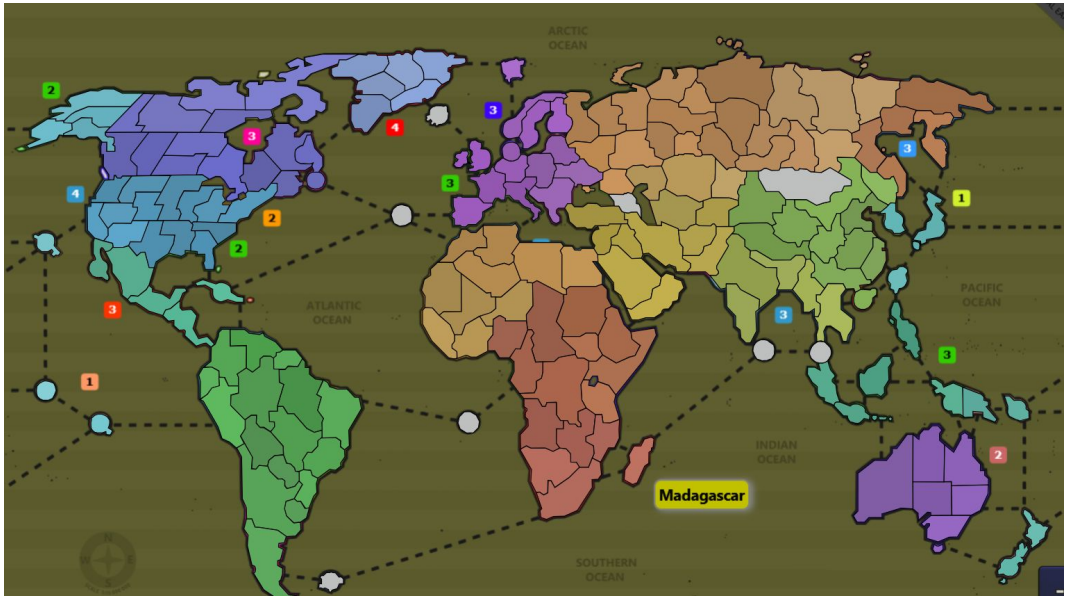
View

```java
public static Map<String, CountryRenderer> loadCountriesRenderers(InputStream inStream) throws Exception {
    Map<String, CountryRenderer> result = new LinkedHashMap<>();

    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
    DocumentBuilder builder = factory.newDocumentBuilder();
    Document doc = builder.parse(inStream);

    NodeList paths = doc.getElementsByTagName("path");
    for (int i = 0; i < paths.getLength(); i++) {
        Element el = (Element) paths.item(i);
        String id = el.getAttribute("id");
        String d  = el.getAttribute("d");
        String style = el.getAttribute("style");
        String fill = parseStyleProperty(style, "fill"); // fill
        String strike = parseStyleProperty(style, "strike"); // border
        Color fillColor = Color.LIGHT_GRAY;
        Color strikeColor = Color.BLACK;
        try {
            if (fill != null) {fillColor = Color.decode(fill);}
            if (strike != null) {strikeColor = Color.decode(strike);}
        } catch (NumberFormatException e) {
            e.printStackTrace();
        }
        if (!id.isEmpty()
            && !d.isEmpty()
            && fillColor!=null
            && strikeColor!=null) {
            CountryRenderer renderer = new CountryRenderer(id, parsePath(d), USE_FILL_COLOR ? fillColor : null
            result.put(id, renderer);
        }
    }

    return result;
```

View (Shape & .svg)

```
    data-country-materials-demand="0"
    data-country-max-facilities="0"
    data-country-building-cost="0"
    data-country-borders="" />
  <path
    style="fill:#569bbd;stroke:#000000"
    d="m 270.83638,254.75975 -21.46595,-0.91344 3.74512,-9.8652 5.38933,0.0913 5
    id="new_england"
    sodipodi:nodetypes="ccccccccccccccccccccc"
    inkscape:label="New England"
    data-country-population="0"
    data-country-troops="0"
    data-country-extraction="0"
    data-country-facilities="0"
    data-country-production="0"
    data-country-materials-demand="0"
    data-country-max-facilities="0"
    data-country-building-cost="0"
    data-country-borders="" />
</g>
<g
   id="nid#south_us"
   inkscape:label="South US"
   style="fill:#bcbcbc;stroke:#000000"
   data-nation-color="#898989"
   data-nation-extraTroops="0"
   data-nation-extraProduction="0"
   data-nation-extraExtraction="0">
  <path
    style="fill:#4d86a3;stroke:#000000"
    d="m 270.83638,254.48573 -49.50871,-1.37018 -2.0236,2.6019 1.39597,1.39189 -
    id="southeast_us"
    sodipodi:nodetypes="cccccccc"
    inkscape:label="Southeast US"
    data-country-population="0"
    data-country-troops="0"
```

View (Shape & .svg)



Dimostrazione, Altro e Domande